

Qué hace el programa

El programa crea una base de datos en MongoDB para un supermercado llamado **Supermercado Base de Datos II**. El sistema permite registrar empleados, categorías de productos, productos disponibles y ventas realizadas.

Además, incluye consultas con Aggregation Pipeline para analizar la información cargada, como ventas por empleado, productos más vendidos, recaudación por día, ventas mayores a determinado importe y recaudación por categoría.

Cómo ejecutarlo

Primero se debe abrir la consola de comandos en la carpeta donde se encuentran los archivos del proyecto.

Para crear la base de datos y cargar los datos iniciales:

```
mongosh supermercado crearBD.js
```

Para ejecutar las consultas:

```
mongosh supermercado consultas.js
```

También se puede ejecutar desde dentro de `mongosh` :

```
use supermercado
load("crearBD.js")
load("consultas.js")
```

Descripción de las colecciones y estructura del modelo

El modelo está compuesto por cinco colecciones:

supermercado Guarda la información general del supermercado.

empleados Contiene los empleados del supermercado. Cada empleado tiene un identificador, nombre, DNI y categoría.

categorias Representa los grupos de productos disponibles. En este modelo existen tres categorías: Almacén, Congelados y Productos de Limpieza.

productos Contiene los productos vendidos por el supermercado. Cada producto tiene

nombre, precio, stock y una referencia a su categoría mediante `categoriaId`.

ventas Registra las ventas realizadas. Cada venta guarda el empleado que la realizó mediante `empleadoId`, la fecha de venta, un arreglo de productos vendidos y el total de la operación.

Dentro de cada venta, los productos vendidos se almacenan embebidos con su `productoId`, nombre, cantidad y precio unitario.

Justificaciones de Diseño

Se utilizó un modelo documental porque permite representar de forma natural la información de una venta, especialmente el detalle de productos vendidos dentro del mismo documento.

Se aplicó **embedding** en la colección `ventas`, guardando los productos vendidos dentro de cada venta. Esta decisión se tomó porque, al consultar una venta, normalmente se necesita ver el ticket completo: fecha, empleado, productos, cantidades, precios y total. Tener esos datos juntos reduce la necesidad de hacer múltiples consultas.

Se aplicó **referencing** entre `ventas` y `empleados`, ya que un empleado puede participar en muchas ventas. Por eso se guarda el `empleadoId` dentro de cada venta.

También se aplicó **referencing** entre `productos` y `categorias`, porque una categoría puede agrupar muchos productos. En lugar de repetir el nombre de la categoría en cada producto, se guarda el `categoriaId`.

Si el modelo se realizara de forma relacional, probablemente habría tablas como empleados, categorías, productos, ventas y detalle_venta. Eso permitiría mayor normalización, pero también requeriría más JOINS para obtener información completa. En MongoDB, en cambio, se aprovecha el documento de venta para guardar el detalle de productos embebido, haciendo más simple la lectura de cada venta.